

AWS Multi-Region VPC Peering and Network Setup

Introduction

Cloud environments rarely exist inside a single isolated network. Modern infrastructures are usually distributed across multiple regions, segmented into public and private networks, and designed with strict access control rules.

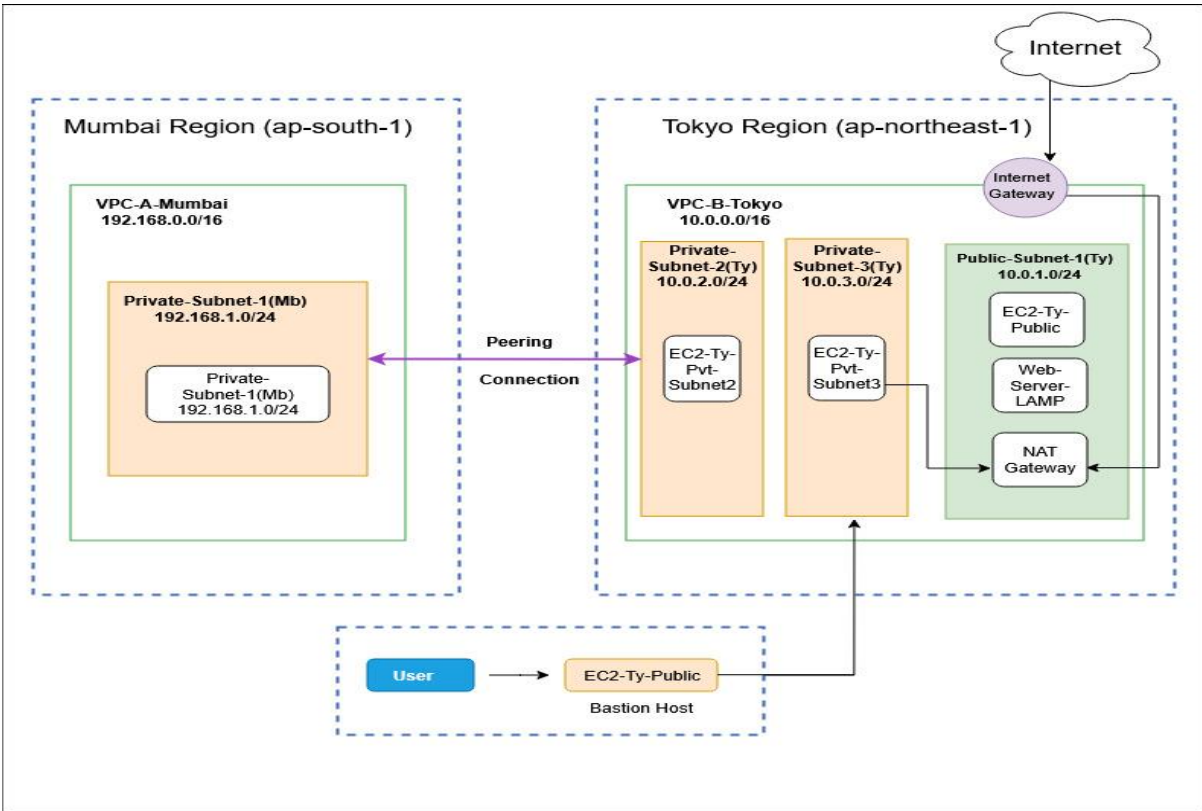
In this project, I built a multi-region AWS networking architecture connecting two VPCs across different regions using VPC Peering, while also implementing secure internet access for private resources using a NAT Gateway. To complete the setup, I deployed a LAMP stack server inside the public subnet.

Project Architecture

The infrastructure spans two AWS regions:

Region	Purpose
Mumbai (ap-south-1)	Private application network
Tokyo (ap-northeast-1)	Public and internal network

Architecture Diagram



This architecture spans two AWS regions and demonstrates how secure communication can be established between isolated cloud networks. The Mumbai VPC contains a private subnet designed for internal communication, while the Tokyo VPC hosts both public and private workloads. Internet access for private resources is managed using a NAT Gateway, and cross-region connectivity is achieved through VPC Peering.

Step 1: Create VPC A in Mumbai Region with Private Subnet

Firstly, switch to region **Mumbai (ap-south-1)** and create the VPC

1. Navigate to **VPC Dashboard** → **Your VPCs**
2. Fill in the details:
 - **Name:** VPC-A-Mumbai
 - **IPv4 CIDR block:** 192.168.0.0/16
3. Click **Create VPC**

The screenshot displays the AWS VPC console for VPC-A-Mumbai (vpc-0f2438f2443cb3abf). The interface includes a top bar with the VPC ID and name, an 'Actions' dropdown, and a 'Details' tab. The details are organized into a grid of key-value pairs:

Details	
VPC ID vpc-0f2438f2443cb3abf	State Available
DNS resolution Enabled	Tenancy default
Main network ACL acl-04f87108ed8d1de44	Default VPC No
IPv4 CIDR (Network border group) -	Network Address Usage metrics Disabled
Encryption control ID -	Encryption control mode -
Block Public Access Off	DHCP option set dopt-0e01ececfa0b9a8
DNS hostnames Disabled	Main route table rtb-0b3faebb2df3b2250
IPv6 CIDR 192.168.0.0/16	IPv6 pool -
Route 53 Resolver DNS Firewall rule groups -	Owner ID 965377249696

After creating the VPC, create a private subnet inside it.

1. In VPC Dashboard, select **Subnets** → **Create Subnet**
2. Configure:
 - **VPC ID:** VPC-A-Mumbai
 - **Subnet name:** Private-Subnet-1(mb)
 - **Availability Zone:** ap-south-1a
 - **IPv4 CIDR block:** 192.168.1.0/24
3. Click **Create Subnet**

The screenshot shows the AWS VPC console's 'Subnets' page. It features a search bar, a filter for 'Subnet ID: subnet-0e6f97a057addf0b0', and a table listing the subnets. The table has columns for Name, Subnet ID, State, VPC, and Block Public. One subnet, 'Private-Subnet-1(mb)', is listed with a state of 'Available' and is associated with VPC-A-Mumbai. The 'Block Public' checkbox is disabled.

Name	Subnet ID	State	VPC	Block Public
Private-Subnet-1(mb)	subnet-0e6f97a057addf0b0	Available	vpc-0f2438f2443cb3abf VPC-A-Mumbai	Off

Since this subnet was intended to remain private, auto-assign public IP was disabled.

For every subnet, a route table has to be created.

1. Go to **Route Tables** → **Create Route Table**
2. **Route Table name:** Private-RT-1(Mb)
3. **VPC:** VPC-A-Mumbai
4. Click **Create**

rtb-0e8f0f89a78c79abf / Private-RT-1(Mb) Actions ▼

Details Info

Route table ID
rtb-0e8f0f89a78c79abf

VPC
vpc-0f2438f2443cb3abf | VPC-A-Mumbai

Main
No

Owner ID
965377249696

Explicit subnet associations
–

Edge associations
–

5. In the route table details, keep the local route (192,168.0.0/16 → Local)
6. Go to **Subnet Association** tab → **Edit Subnet Associations**
 - Check the ‘Private-Subnet-1(Mb)’

VPC > Route tables > rtb-0e8f0f89a78c79abf > Edit subnet associations

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/1)
Filter subnet associations

☒	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	Private-Subnet-1(mb)	subnet-0e6f97a057addf0b0	192.168.1.0/24	–	Main (rtb-0b3faebb2df3b2250)

Selected subnets

subnet-0e6f97a057addf0b0 / Private-Subnet-1(mb) X

Cancel Save associations

Note: This subnet has no internet gateway attachment, making it truly private.

Step 2: Create VPC B in Tokyo Region with Three Subnets

Switch to region **Tokyo (ap-northeast-1)** and create a VPC.

1. Navigate to **VPC Dashboard** → **Create VPC**
2. Configure:
 - **Name:** VPC-B-Tokyo
 - **IPv4 CIDR block:** 10.0.0.0/16
3. Click **Create VPC**

vpc-0a12c3052b1d64c8b / VPC-B-Tokyo

Actions

Details

Info

VPC ID

vpc-0a12c3052b1d64c8b

DNS resolution

Enabled

Main network ACL

acl-051f342fb11ec6aa7

IPv6 CIDR (Network border group)

-

Encryption control ID

-

State

Available

Tenancy

default

Default VPC

No

Network Address Usage metrics

Disabled

Encryption control mode

-

Block Public Access

Off

DHCP option set

dopt-05df71b952485598d

IPv4 CIDR

10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups

-

DNS hostnames

Disabled

Main route table

rtb-06b6f06309384357b

IPv6 pool

-

Owner ID

965377249696

Since the Tokyo VPC would contain public-facing resources, internet connectivity needed to be configured using an Internet Gateway. It is required for public subnet communication with the internet.

1. Go to **Internet Gateways** → **Create Internet Gateway**
2. **Name:** IGW-Tokyo
3. Click **Create**

igw-0da993389a70eb364 / IGW-Tokyo

Actions

Details

Info

Internet gateway ID

igw-0da993389a70eb364

State

Detached

VPC ID

-

Owner

965377249696

Tags (1)

Manage tags

Search tags

Key	Value
Name	IGW-Tokyo

4. Select the created IGW, then **Attach to VPC**
5. Select **VPC-B-Tokyo** and confirm

VPC > Internet gateways > Attach to VPC (igw-0da993389a70eb364)

Attach to VPC (igw-0da993389a70eb364)

VPC

Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.

Available VPCs

Attach the internet gateway to this VPC.

vpc-0a12c3052b1d64c8b

AWS Command Line Interface command

Cancel

Attach internet gateway

We have to create three subnets – one public subnet and two private subnets.

Subnet 1

Subnet 1 of 3

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block
 256 IPs

< > ^ v

Enable auto-assign public IP for public subnet of VPC B.

Edit subnet settings [Info](#)

Subnet

Subnet ID  subnet-0d46165bdd47ef615	Name  Public-Subnet-1(Ty)
--	--

Auto-assign IP settings [Info](#)
Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ **Enable auto-assign public IPv4 address** [Info](#)

☐ **Enable auto-assign customer-owned IPv4 address** [Info](#)
Option disabled because no customer owned pools found.

Subnet 2

Subnet 2 of 3

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block
 256 IPs

< > ^ v

Subnet 3

Subnet 3 of 3

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

Private-Subnet-3(Ty)

The name can be up to 256 characters long.

Availability Zone

Info

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block

Info

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.3.0/24

256 IPs

< > ^ v

Create Route Tables for all the subnets.

Public Route Table

1. Create **Route Table** → **Public-RT-Tokyo**
2. **VPC:** VPC-B-Tokyo

rtb-0e2d21d637af5adc8 / Public-RT-Tokyo

Actions

Details

Info

Route table ID

rtb-0e2d21d637af5adc8

VPC

vpc-0a12c3052b1d64c8b | VPC-B-Tokyo

Main

No

Owner ID

965377249696

Explicit subnet associations

-

Edge associations

-

3. Go to **Routes** tab:
 - Add route:
 - **Destination:** 0.0.0.0/0
 - **Target:** Internet Gateway (IGW-Tokyo)

Edit routes

Destination

10.0.0.0/16

Target

local

Q local

Internet Gateway

Q igw-0da993389a70eb364

Status

Active

-

Propagated

No

No

Route Origin

CreateRouteTable

CreateRoute

Add route

Remove

Cancel

Preview

Save changes

4. Associate with **Public-Subnet-1(Ty)**

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/3)

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	Private-Subnet-2(Ty)	subnet-0e21343325a09b4d1	10.0.2.0/24	–	Main (rtb-06b6f06309384357b)
☒	Public-Subnet-1(Ty)	subnet-0d46165bdd47ef615	10.0.1.0/24	–	Main (rtb-06b6f06309384357b)
☐	Private-Subnet-3(Ty)	subnet-05b934b0693999dd8	10.0.3.0/24	–	Main (rtb-06b6f06309384357b)

Private Route Table for Subnet 2

- Create **Route Table** → **Private-RT-2(Ty)**
- VPC:** VPC-B-Tokyo

rtb-04d8341d7a150343e / Private-RT-2(Ty) Actions

Details [Info](#)

Route table ID
 rtb-04d8341d7a150343e

VPC
 vpc-0a12c3052b1d64c8b | VPC-B-Tokyo

Main
 No

Owner ID
 965377249696

Explicit subnet associations
–

Edge associations
–

3. Associate with **Private-Subnet-2(Ty)**

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/3)

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☒	Private-Subnet-2(Ty)	subnet-0e21343325a09b4d1	10.0.2.0/24	–	Main (rtb-06b6f06309384357b)
☐	Public-Subnet-1(Ty)	subnet-0d46165bdd47ef615	10.0.1.0/24	–	rtb-0e2d21d637af5adc8 / Public-RT-T...
☐	Private-Subnet-3(Ty)	subnet-05b934b0693999dd8	10.0.3.0/24	–	Main (rtb-06b6f06309384357b)

Private Route Table for Subnet 3

- Create **Route Table** → **Private-RT-3-NAT(Ty)**
- VPC:** VPC-B-Tokyo

rtb-07baab6dde55f816e / Private-RT-3-NAT(Ty) Actions

Details [Info](#)

Route table ID
 rtb-07baab6dde55f816e

VPC
 vpc-0a12c3052b1d64c8b | VPC-B-Tokyo

Main
 No

Owner ID
 965377249696

Explicit subnet associations
–

Edge associations
–

3. Add route:
 - **Destination:** 0.0.0.0/0
 - **Target:** NAT Gateway (refer to Step 3)
4. Associate with **Private-Subnet-3(Ty)**

Edit subnet associations
Change which subnets are associated with this route table.

Available subnets (1/3)

Filter subnet associations

	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR	Route table ID
☐	Private-Subnet-2(Ty)	subnet-0e21343325a09b4d1	10.0.2.0/24	-	rtb-04d8341d7a150343e / Private-RT-..
☐	Public-Subnet-1(Ty)	subnet-0d46165bdd47ef615	10.0.1.0/24	-	rtb-0e2d21d637af5adc8 / Public-RT-T..
☒	Private-Subnet-3(Ty)	subnet-05b934b0693999dd8	10.0.3.0/24	-	Main (rtb-06b6f06309384357b)

Step 3: Configuring NAT Gateway for Private Subnet Internet Access

NAT Gateway allows Subnet 3 to initiate outbound internet connections while blocking inbound.

1. Go to **NAT Gateways** → **Create NAT Gateway**
2. Configure:
 - **Subnet:** Public-Subnet-1(Ty)
 - **Elastic IP allocation:** Click **Allocate Elastic IP**
 - **Name:** NAT-Gateway-Public
3. Wait for status to change to 'Available'

nat-05fc78da07e4b3d43 / NAT-Gateway-Public Actions

Details

NAT gateway ID nat-05fc78da07e4b3d43	Connectivity type Public	State Pending	State message Info -
NAT gateway ARN arn:aws:ec2:ap-northeast-1:965377249696:natgateway/nat-05fc78da07e4b3d43	Primary public IPv4 address -	Primary private IPv4 address -	Primary network interface ID -
VPC vpc-0a12c3052b1d64c8b / VPC-B-Tokyo	Subnet subnet-0d46165bdd47ef615 / Public-Subnet-1(Ty)	Created Thursday, May 21, 2026 at 11:53:56 GMT+5:30	Deleted -

4. Edit **Private-RT-3-NAT(Ty)** route table:
 - Add route
 - **Destination:** 0.0.0.0/0
 - **Target:** NAT Gateway

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	NAT Gateway	-	No	CreateRoute

[Add route](#) [Cancel](#) [Preview](#) [Save changes](#)

Step 4: Security Groups Configuration

VPC-A Mumbai Security Group (Mumbai Region)

1. Go to **EC2** → **Security Groups** → **Create Security Group**
2. **Name:** SG-VPC-A-Private
3. **VPC:** VPC-A-Mumbai
4. **Inbound Rules:**
 - Type: SSH, Port: 22, Source: Anywhere IPv4
 - Type: ALL ICMP – IPv4, Source: Anywhere IPv4
5. **Outbound Rules:** default

sg-09f2c78d22e5acb70 - SG-VPC-A-Private Actions ▼

Details

Security group name SG-VPC-A-Private	Security group ID sg-09f2c78d22e5acb70	Description Allows SSH and ICMP for peering connection	VPC ID vpc-0b9967b1715ffa936
Owner 965377249696	Inbound rules count 3 Permission entries	Outbound rules count 1 Permission entry	

VPC-B Tokyo – Public Subnet Security Group (Tokyo Region)

1. **Name:** SG-VPC-B-Public
2. **VPC:** VPC-B-Tokyo
3. **Inbound Rules:**
 - Type: SSH, Port: 22, Source: Anywhere IPv4
 - Type: HTTP, Port: 80, Source: Anywhere IPv4
 - Type: HTTPS, Port: 443, Source: Anywhere IPv4
4. **Outbound Rules:** default

sg-0ab8a5fd75c5294b1 - SG-VPC-B-Public Actions ▼

Details

Security group name SG-VPC-B-Public	Security group ID sg-0ab8a5fd75c5294b1	Description Allows SSH	VPC ID vpc-0a12c3052b1d64c8b
Owner 965377249696	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

VPC-B Tokyo – Private Subnets Security Group (Tokyo Region)

1. **Name:** SG-VPC-B-Private
2. **VPC:** VPC-B-Tokyo
3. **Inbound Rules:**
 - Type: SSH, Port: 22, Source: 0.0.0.0/0
 - Type: ALL ICMP – IPv4, Protocol: ICMP, Source: 0.0.0.0/0
4. **Outbound Rules:** default

sg-09624643691a97874 - SG-VPC-B-Private

Actions

Details

Security group name

SG-VPC-B-Private

Security group ID

sg-09624643691a97874

Description

Allows SSH

VPC ID

[vpc-0a12c3052b1d64c8b](#)

Owner

965377249696

Inbound rules count

3 Permission entries

Outbound rules count

1 Permission entry

Step 5: Launching EC2 Instances

Instance 1: VPC-A Mumbai Private Subnet (Mumbai Region)

- Go to **EC2** → **Instances** → **Launch Instances**
- Name:** EC2-Mb-Pvt-Subnet
- AMI Selection:** Amazon Linux 2023 (kernel-6.1)
- Instance Details:**
 - Instance Type:** t3.micro
 - VPC:** VPC-A-Mumbai
 - Subnet:** Private-Subnet-1(Mb)
 - Auto-assign Public IP:**
- Security Group:** SG-VPC-A-Private
- Key Pair:** Select or create your key pair
- Review and **Launch Instance**

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

EC2-Mb-Pvt-Subnet

Clear filters

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☐	EC2-Mb-Pvt-Su...	i-0c56abdb40b1b63de	Running	t3.micro	Initializing	ap-south-1a	-

Instance 2: VPC-B Tokyo Private Subnet 2 (Tokyo Region)

- Name:** EC2-Ty-Pvt-Subnet2
- VPC:** VPC-B-Tokyo
- Subnet:** Private-Subnet-2(Ty)
- Security Group:** SG-VPC-B-Private

Instances (1) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☐	EC2-Ty-Pvt-Su...	i-047ae4016d2a72f6c	Running	t3.micro	Initializing	ap-northeast-1a	-

Instance 3: VPC-B Tokyo Private Subnet 3 (Tokyo Region)

1. **Name:** EC2-Ty-Pvt-Subnet3
2. **VPC:** VPC-B-Tokyo
3. **Subnet:** Private-Subnet-3(Ty)
4. **Security Group:** SG-VPC-B-Private

Instances (1)

Info

Last updated less than a minute ago

Connect

Instance state ▾

Actions ▾

Launch instances ▾

Saved filter sets

Choose filter set ▾

Find Instance by attribute or tag (case-sensitive)

EC2-Ty-Pvt-Subnet3

×

Clear filters ▾

<

1

>

☐	Name	Instance ID	Instance state ▾	Instance type ▾	Status check	Availability Zone ▾	Public IP
☐	EC2-Ty-Pvt-Su...	i-07ff47431dd230bbb	Running	t3.micro	Initializing	ap-northeast-1a	-

Instance 4: VPC-B Tokyo Public Subnet (Tokyo Region)

1. **Name:** EC2-Ty-Public
2. **VPC:** VPC-B-Tokyo
3. **Subnet:** Public-Subnet-1
4. **Security Group:** SG-VPC-B-Public

Instances (1) Info

Last updated less than a minute ago

Connect

Instance state ▾

Actions ▾

Launch instances ▾

Saved filter sets

Choose filter set ▾

Find Instance by attribute or tag (case-sensitive)

EC2-Ty-Public X

Clear filters ▾

< 1 > ⚙

☐	Name  ▾	Instance ID	Instance state ▾	Instance type ▾	Status check	Availability Zone ▾	Public IP
☐	EC2-Ty-Public	i-0ac94e6e91390bdf8	 Running  	t3.micro	 Initializing	ap-northeast-1a	-

Step 6: Establishing Cross-Region VPC Peering

Switch to **Mumbai** region

1. Navigate to **VPC Dashboard** → **Peering Connections**
2. Configure the following:
 - **Name:** Mumbai-Tokyo-Peering
 - **Requester VPC:** VPC-A-Mumbai
 - **Account:** My Account
 - **Region:** ap-northeast-1 (Tokyo)
 - **Accepter VPC:** VPC-B-Tokyo

3. Click Create Peering Connection

pcx-0dced9d2e0ebe3f39 / Mumbai-Tokyo-Peering Actions ▾

Details Info

Requester owner ID
965377249696

Peering connection ID
pcx-0dced9d2e0ebe3f39

Status
Initiating Request to 965377249696

Expiration time
Thursday, May 28, 2026 at 11:35:42 GMT+5:30

Accepter owner ID
965377249696

Requester VPC
vpc-0f2438f2443cb3abf / VPC-A-Mumbai

Requester CIDRs
192.168.0.0/16

Requester Region
Mumbai (ap-south-1)

VPC Peering connection ARN
arn:aws:ec2:ap-south-1:965377249696:vpc-peering-connection/pcx-0dced9d2e0ebe3f39

Accepter VPC
vpc-0a12c3052b1d64c8b

Accepter CIDRs
-

Accepter Region
Tokyo (ap-northeast-1)

The peering connection status initially appears as **Pending Acceptance**.

Since this was a cross-region peering setup, the request had to be accepted from the Tokyo region. Switch to **Tokyo** region.

1. Navigate to **VPC Dashboard** → **Peering Connections**
2. Select **Mumbai-Tokyo-Peering**
3. Select **Action** → **Accept Request**

Peering connections (1/1) Info Actions ▴ Create peering connection

Name	Peering connection ID	Status	Accepter VPC
-	pcx-0dced9d2e0ebe3f39	Pending acceptance	vpc-0a12c3052b1d64c8b

View details

Accept request

Reject request

Edit DNS settings

Manage tags

Delete peering connection

Once accepted, the peering status changes to **Active**.

The communication would still not work until the route tables are updated.

In **Mumbai** region,

1. Go to **Private-RT-1(Mb)** route table (VPC-A)
2. Add route:
 - **Destination:** 10.0.2.0/24 (Private Subnet 2 in Tokyo)
 - **Target:** Peering Connection

Edit routes

Destination	Target	Status	Propagated	Route Origin
192.168.0.0/16	local	Active	No	CreateRouteTable
10.0.2.0/24	Peering Connection	-	No	CreateRoute

local

Peering Connection

pcx-0dced9d2e0ebe3f39

Add route

Cancel

Preview

Save changes

In **Tokyo** region,

3. Go to **Private-RT-2(Ty)** route table (VPC-A)
4. Add route:
 - **Destination:** 192.168.1.0/24 (Private Subnet 1 in Mumbai)
 - **Target:** Peering Connection

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	CreateRouteTable
192.168.1.0/24	Peering Connection	-	No	CreateRoute

Step 7: Testing Peering Connection

Since the Mumbai EC2 instance resided inside a private subnet without public internet access, the Tokyo public EC2 instance was used as a bastion host to access the private environments securely.

1. Connect to **EC2-Ty-Public**
2. Create a file `key.pem` using the following command
`$ nano key.pem`

Copy the contents of Key pair of Tokyo to this file and save.

```
GNU nano 2.9.3 key.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA...
-----END RSA PRIVATE KEY-----

i-0ac94e6e91390bdf8 (EC2-Ty-Public)
PublicIPs: 57.180.64.98 PrivateIPs: 10.0.1.133
```

3. The key pair file permissions were updated
`$ chmod 400 key.pem`
4. Connect to the private instance of Tokyo region using its private instance IP:
`$ ssh -i key.pem ec2-user@10.0.2.8`

```
[root@ip-10-0-1-133 ec2-user]# ssh -i key.pem ec2-user@10.0.2.8
The authenticity of host '10.0.2.8 (10.0.2.8)' can't be established.
ED25519 key fingerprint is SHA256:IzEodH9hw+rsGkGSjkRDfclDP5Ir+fx3ybFUyYu0dB4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.0.2.8' (ED25519) to the list of known hosts.
```

```
      #_
    ~\####         Amazon Linux 2023
   ~~\_#####\
   ~~ \###|
   ~~ \#/        https://aws.amazon.com/linux/amazon-linux-2023
     ~~~~~V~'--->
       ~~~
       ~~-.-
          /_/_/_/_/_
           /m/'
```

```
[ec2-user@ip-10-0-2-8 ~]$
```

5. Connect to the private instance of Mumbai region using its private instance IP
`$ ssh -I key.pem ec2-user@192.168.2.223`

```
[ec2-user@ip-10-0-2-8 ~]$ ssh -i key.pem ec2-user@192.168.1.223
Warning: Identity file key.pem not accessible: No such file or directory.
The authenticity of host '192.168.1.223 (192.168.1.223)' can't be established.
ED25519 key fingerprint is SHA256:I6uf3zSwzapudAJZXf0YRRysIm0tqWYdMvl0SmZVZms.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.1.223' (ED25519) to the list of known hosts.
ec2-user@192.168.1.223: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

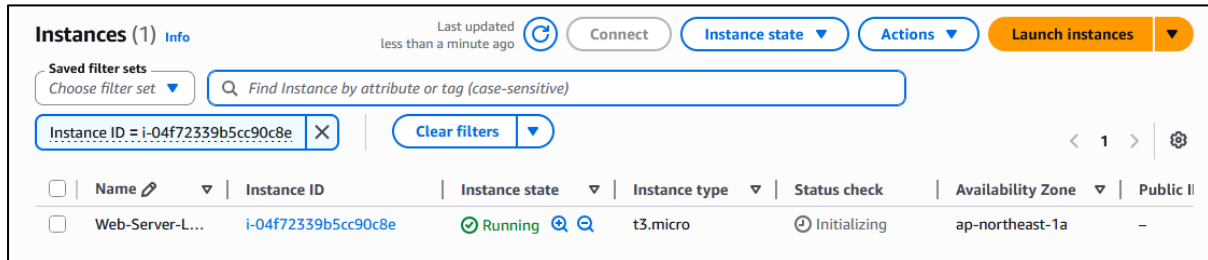
6. Once connected, test communication.

```
[ec2-user@ip-10-0-2-8 ~]$ ping 192.168.1.223
PING 192.168.1.223 (192.168.1.223) 56(84) bytes of data.
64 bytes from 192.168.1.223: icmp_seq=1 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=2 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=3 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=4 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=5 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=6 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=7 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=8 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=9 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=10 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=11 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=12 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=13 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=14 ttl=127 time=127 ms
64 bytes from 192.168.1.223: icmp_seq=15 ttl=127 time=127 ms
```

Step 8: Deploying the LAMP Stack Server

1. Launch a new instance in VPC B in Subnet 1 with the following configurations:
 - **Name:** Web-Server-LAMP

- **AMI:** Ubuntu Server 26.04 LTS
- **Instance Type:** t3.micro
- **VPC:** VPC-B-Tokyo
- **Subnet:** Public-Subnet-1(Ty)
- **Auto Assign Public IP:** Enabled
- **Security Group:** SG-VPC-B-Public

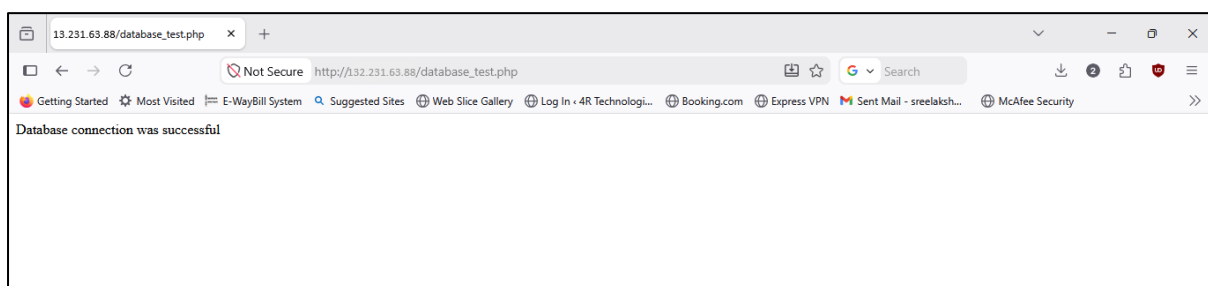


2. After launching the instance, connect to the server and complete the LAMP stack installation and configuration.

For the detailed step-by-step setup process, including Apache, MySQL, PHP installation, and database connectivity configuration, refer to the following guide: [LAMP Stack Installation Guide](#)

3. After completing the setup, verify the deployment by accessing the following outputs from the browser:

`https://Public_IP//database_test.php`



Conclusion

This project provided a hands-on understanding of how AWS networking components work together in a real-world cloud environment. From configuring multi-region VPCs and secure subnet communication to implementing NAT Gateway routing, VPC peering, bastion host access, and deploying a LAMP stack server, the setup demonstrated the practical side of building secure and scalable cloud infrastructure on AWS.